

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_4849400f-9d3\agent-ad422caed37eeb9d9.jsonl`  
- SHA-256: `f6af6eac38e49887336b0ec1c437d7cea7fa11ed5d533efe05bd2872f82d782b`  
- Source modified: `2026-06-21T09:27:49+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_4849400f-9d3`  
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T09:24:52.594Z)

You are mapping OWNER-GATED decisions for the badminton-highlight-indexer ENGINE serving track (project doc docs/COMPUTE_DECOUPLED_SERVING/README.md). Context: M0-M4 are SHIPPED (the config/input/output/device SEAM layer: deployment.profile+presets, input_backend, output base, DeviceContext). NOW the team is on M5+. The owner wants to BATCH-DECIDE every gate so the maximum independent code path is unblocked. The discipline: ONE isolated PR per milestone off origin/master, default-OFF + golden/cross-OS-gated for any core-caller change, mocked tests for anything not runnable locally (no GPU/cloud here). KEY DISTINCTION you must draw for every gate: separate (a) CODE that CAN be written + merged NOW behind default-OFF with mocked/faked tests (no owner needed) from (b) the precise OWNER PREREQUISITE (real GPU run, GCP spend, counsel/consent review, real prices, OAuth creds, a design pick). For each gate return the structured fields. Ground every claim in real file:line or doc anchors ? read the code, do not guess. Be exhaustive within your area but do not stray outside it.

YOUR AREA: the serving milestones M5-M12 and the \$8 open design questions, from the project doc itself.

READ: docs/COMPUTE_DECOUPLED_SERVING/README.md (\$2 decisions D1-D15, \$4 design, \$7 tracker M5-M12, \$8 open questions, \$9 DoD), docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md, and ADR-014 in docs/archives/decisions/DECISIONS.md.

For EACH of M5,M6,M7,M8,M9,M10,M11,M12 extract its gate. ALSO extract each STILL-OPEN \$8 question (Q2 stitch placement, Q8 NVDEC/NVENC vs libx264, Q9 input cap <2GB-vs-4GB, Q10 dir name) as a design_choice gate ? these are cheap owner picks that unblock M6/M7 code. Note dependencies between milestones (M6 deps M5; M7 deps M6; M8/M9 dep M7; M11 deps M7,M9; M12 deps M7,M9).

2. assistant (2026-06-21T09:24:56.339Z)

I'll map the owner-gated decisions for the serving milestones M5-M12 and the \$8 open design questions. Let me start by reading the relevant docs.

3. user (2026-06-21T09:24:59.466Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context' from the remote, but no such ref was fetched.

4. user (2026-06-21T09:24:59.900Z)

```
1      # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage  
2  
3      > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .  
**Owner:** Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21  
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to  
`docs/archives/past_projects/` when DONE)  
5      > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory  
`current-state` + `docs/NEXT_STEPS.md` once work starts.
```

6 > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED\_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).

7 > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
compute-decoupled serving because **truly-local** is a co-equal, first-class profile, not an
afterthought.

8 > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf\_56274ff0`), `[design]` (proposed), `[researched]`.

9

10 ---

11

12 > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive**, next
to the source. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect **suggests**, the user
confirms (D15).

13

14 ## 0. TL;DR

15 The rally engine is a **pure function** of `(input bytes + config) ? outputs` in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile** may branch
on. A **ComputeBackend** (deployment profile) decides **where** the core runs (a local process
vs a Cloud Run GPU job); a **StorageBackend** decides **where bytes come from** (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus
cpu-mock for CI. The single most important caveat: detection is **byte-identical** only on the
same GPU (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the
parity guarantee is enforced at the WINDOW + STITCH layer, not detector-CSV bytes.

16

17 ## 1. Problem & goal

18 - **Why now:** DECOUPLED_COMPUTE proved compute **portability** (M0?M2 + M3.5 on a GCP L4)
and is archived; its deferred **M3** (serving endpoint + per-video billing) + M4 (scale-to-zero +
cost guard) become this project. The owner needs **both** a hosted service **and** a truly-local
mode, with the **core** (detect+stitch) unaffected by which one runs.

19 - **The two user-facing outcomes** (owner, 2026-06-21):

20 - **(a) Hosted service** ? operate on small **local uploads** (`<2GB`) **and** load from
gdrive / a bucket, spinning up **Cloud Run** for both rally segmentation AND stitching
(stitching is compute-intensive ? run it where its cost is accounted for).

21 - **(b) Truly-local** ? on a local machine with videos on **local drives/USB** + a local
GPU, run inference **and** stitching **completely locally**.

22 - **"Good"** looks like: the same `video` ? rallies ? reel` core gives identical windows
+ stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup**
choice, never a code branch in the core.

23

24 ## 2. Decisions locked

25

#	Decision	Source	Implication
D1	The core is a pure 3-stage function (DETECT/WINDOW/STITCH); no profile branch inside it. design `[verified]` All profile difference lives in config + the two registries.		
D2	Two registries: `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`. ADR-014 Realizes PLATFORM_ARCHITECTURE's `local`/`hosted`/`byo_worker`.		
D3	Profiles shipped: `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred. ADR-014 Enum/seam reserved so BYO isn't precluded.		
D4	In cloud-serving , STITCH runs on Cloud Run (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. owner (a) Stitch-on-laptop would hide real cost.		
D5	Parity enforced at WINDOW+STITCH (byte-exact for a fixed trajectory); detect is byte-exact only same-GPU. deploy report 2026-06-20 Cross-GPU asserted at the rally/window level, never CSV bytes.		
D6	Default-OFF , smallest-safe-first, one PR per milestone; any core-caller		

change (e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-convention]] | Zero-regression discipline. |

34 | D7 | ****Cloud Run cold-start: SCALE-TO-ZERO**** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). ****No warm pool**** (that ~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |

35 | D8 | ****Pricebook = a versioned DB table****; cost computed in ****USD**** (matches GCP billing = one source of truth), ****displayed in INR**** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |

36 | D9 | ****Edge auth = Cloudflare Access**** (reuse the site's existing CF). Lock the Cloud Run ****ingress to the CF proxy**** + ****verify the `Cf-Access` JWT signature**** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |

37 | D10 | ****Free-tier "data-for-reels" trade:**** free reels in exchange for ****training rights**** (uploaded footage + derived trajectories/labels); ****paid tier = no-train**** (privacy is the paid feature). ****Carve-outs:**** train ONLY on attested ****ADULT**** footage (minor footage ? reel yes, training pool ****NO**** ? DPDP/GDPR need verifiable parental consent); train on ****DERIVED**** data + ****auto-delete raw****; ****ToS counsel-reviewed****; live training-on-uploads ****gated to M9**** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel. |

38 | D11 | ****Quality floor: Gemini handoff ON**** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a ****Launch Report****; the owned model runs in ****shadow/eval**** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |

39 | D12 | ****Data-flywheel harness is IN SCOPE** (owner add, Q5): ****capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store**** in the per-video workspace/bucket ? ****run shadow evals**** (owned-vs-Gemini, dual-eval-set) ? ****promote good ones to the golden TRAINING set**** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new ****M11****; ties D10 (consent) + `data\_pipeline/` + the eval/experiment harness. |

40 | D13 | ****Profile is user-switchable ANYTIME ? a headline FEATURE.**** The same user runs ****local today, cloud tomorrow, back to local**** ? it's just a config/startup choice; the pure core gives ****equivalent**** output either way. ****Advertise it**** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the ****rally/window**** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is ****between**** jobs. |

41 | D14 | ****NORTHSTAR ? operate toward this:**** a ****mobile, app-like**** flow ? point at a ****Google Drive**** video ? ****fully cloud-native**** run ? the stitched, ****ready-to-share**** reel lands ****back in Drive, next to the source****. | owner 2026-06-21 | Implies a ****`gdrive-api` (OAuth) StorageBackend**** (read source + write reel back) ? distinct from the local ****mount**** backend; ****resolves \$8 #7****, new ****M12****. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the ****`StorageBackend`** ABC accommodates it). |

42 | D15 | ****Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.**** A GPU owner may still choose cloud; ****cloud (= spend) is never entered silently****; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |

43

44 ## 3. Foundation ? evolution / ADR lineage ? ****trace the idea end-to-end****

45 This project is the latest step in a long decision chain; each ADR contributed a piece and a ****subtlety that carries forward****:

46

47 | ADR / doc | Contributed | Subtlety carried into this project |

48 |---|---|---|

49 | ****ADR-005**** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the ****container image**** (ffmpeg, fonts, weights) stay commercial-clean. |

50 | ****ADR-008**** | Owned-model topology; ****train==serve featurizer single-sourcing**** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to ****local/cloud****. |

51 | ****ADR-009**** | KhelSutra Guru; ****local-first delivery**** | The ****truly-local profile**** is the direct continuation ? privacy/offline self-host stays first-class. |

52 | ****ADR-010**** | DECOUPLED: D0?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |

53 | ****ADR-011**** | DECOUPLED scope = M0?M2+M3.5; ****deferred M3 (serving) + M4**

```

(billing/cost-guard)**; override the "rent nothing / not a service" posture | **This project
realizes M3/M4.** Carries $06's owner-gated gates: **DPA/consent for non-owner uploads,
collector `md5` keying, WASB-C3**. |
54 | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving
model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope
this expands. |
55 | **ADR-013** | Drop D0; **GCP sole compute stack**; $200 D0 credits ? non-compute only
| The provider for cloud-serving; D0 Spaces remains only a possible S3 *storage* target. |
56 | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected
compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not**
supersede them. |
57 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The
abstraction this realizes in code. |
58 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop`
backend | The truly-local profile = these, productionized. |
59 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB
hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
60 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel
parity finding** | Why D5 (parity at window level, not CSV bytes). |
61
62 ## 4. Design / architecture
63 See [architecture.svg](architecture.svg) for the picture. **The core never changes;
the box around it does.**
64
65 ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
66 1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ?
CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable
impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ?
emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
67 2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?)
? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same
trajectory + config = byte-identical candidate windows. Used by every runner *and* the
eval/regression stack verbatim.
68 3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments,
out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffmpeg ? watermark
? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
69
70 **Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on
profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-
identical to CONTROL).
71
72 ### 4.2 The deployment profiles
73 | Profile | Compute | Storage | Orchestration | When |
74 |---|---|---|---|---|
75 | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto`
(WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main`
or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-
box ? owner (b) |
76 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda);
**detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload);
per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job
pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted
SaaS ? owner (a) |
77 | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) |
`local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free
dev |
78 | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket
(signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ?
**DEFERRED**, enum/seam reserved |
79
80 ### 4.3 Profile selection (the "startup/setup config") `[design]`
81 ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-
detect in `load_config`:
82 - `deployment.profile` {truly-local | cloud-serving | cpu-mock} (empty = today's
manual knobs, **fully backward-compatible**).

```

83 - Selecting a profile applies a **coherent preset bundle** of **existing** knobs that today drift independently: **INPUT** (new `input_backend`/input_root``, parallel to the output-side `StorageConfig``), **COMPUTE** (new `compute_target`` enum locking `detector_impl`` + `skip_ai_handoff`` + workspace strategy), **STORAGE** (`storage.backend`/workspace_root`/single_folder_per_video``, all already exist).

84 - **Precedence**: built-in defaults ? profile preset ? `config.json`` ? env (`RALLY_DETECTOR_IMPL``, `WASB_*`, `DEPLOYMENT_PROFILE``, ?) ? worker `--set k=v``. Every knob stays individually overridable.

85 - **Env auto-detect SUGGESTS**, the user **CONFIRMS** (D15 ? no surprise execution): the env gives a **proposed** default (`!K_SERVICE`` ? cloud-serving; `!CI=true`` ? cpu-mock; else `!torch.cuda.is_available()`` ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).

86 - The worker (`cmd_infer`/cmd_train``) is **already profile-agnostic** (reads `config.storage``, accepts `--config/--set``) ? no worker rewrite.

87

88 **### 4.4 Compute placement ? and why stitch runs on Cloud Run** `[design]``

89 DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving**, stitch is co-located in the Cloud Run job so `gpu_active_seconds`` (detect) + `wall_seconds`` (detect+stitch) + `bytes_out`` (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim_due_job`` already supports both.)

90

91 **### 4.5 Reuse map ? most of the seams already exist** `[verified]``

92 **Exists**: the 3-stage core; `_resolve_runner`` (the single compute seam); `StubDetectorRunner`` (`replay_csv`` = byte-exact \$0 anchor); `StorageBackend`` ABC + 4 backends + `StorageRef.parse_uri``; the worker CLI (`fetch/put``, `--config/--set``); the async job control plane (`jobs`` table, `claim_due_job`` CAS); `ExecutionPolicy WAIT_AND_RESUME`/JobWaiting`` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB``); `skip_ai_handoff`` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial**: per-video workspace helper (default-OFF); `DeviceContext`` (designed); edge-auth (v4 `user_id`` columns exist). **Net-new**: the `deployment`` section; `input_backend``; configurable output base (kill hardcoded `output/``); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.

93

94 **### 4.6 The data-flywheel harness** (D12 / M11) `[design]``

95 Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:

96 1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (`workspaces/<video_id>/``), instead of the no-consent auto-delete path.

97 2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).

98 3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (`backend/eval/experiment.compare`` + the dual-eval-set, `[[eval-dual-set-regression]]`) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").

99 4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? `data_pipeline/``), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.

100 This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data_pipeline/``, `backend/eval/``, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.

101

102 **## 6. Honest limits ? what this does NOT do / prove**

103 - A green CI suite proves **plumbing + determinism** (window+stitch) + **profile-parity** for \$0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).

104 - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.

105 - The cloud `0.1.0-l4`` weights are **n=2 plumbing-grade** ? cloud-serving ships with

****Gemini handoff ON**** until the owned model clears a floor (open Q).

106 - This does not resolve ****WASB-C3**** (sharing a trained model) or the ****DPA/consent**** gate ? both remain owner/counsel-gated before public, non-owner serving.

107

108 **## 7. Deliverables & progress tracker ? **source of truth****

109 Legend: ? Todo · ? In progress · ? Done · ? Gated. ****One small PR per row****, off `origin/master`, no stacking. Default-OFF where it touches core callers.

110

111 | ID | Deliverable | Depends | Gated? | Status | PR |

112 |----|-----|-----|-----|-----|----

113 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? |

****? APPROVED 2026-06-21**** |

114 | M1 | `deployment.profile` + `compute\_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? |

[#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |

115 | M2 | `input\_backend`/`input\_root`; wire main CLI/API input through `StorageRef.parse\_uri` (local=identity); storage-fake tests | M1 | safe | ? |

[#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |

116 | M3 | Configurable output/scratch base ? kill hardcoded `output/` (`trajectory\_hybrid:237,313`, `yolo\_hybrid:407`, `gemini`); ****DEFAULT stays `output/`****; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |

[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |

117 | M4 | `DeviceContext` + WASB ****CPU-fallback****; unified healthcheck/device wiring | M1 | safe (cuda default unchanged) | ? |

[#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283) |

118 | M5 | ****truly-local profile**** end-to-end (preset + startup checks); first committed ****runlog**** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | ? |

119 | M6 | Cloud Run GPU ****dispatch shim**** (control plane ? job via storage ? re-claim) + ****containerized worker image**** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | ? |

120 | M7 | ****cloud-serving profile**** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; ****DEPLOY_REPORT**** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |

121 | M8 | Per-job ****cost ledger**** (v_ migration: `owner\_id, gpu\_active\_seconds, wall\_seconds, bytes\_out, cost\_usd, cost\_breakdown\_json`) + ****pricebook**** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | ? |

122 | M9 | ****Edge auth**** (Cf-Access extraction) + per-tenant scoping on `user\_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |

123 | M10 | `byo\_worker` / zero-infra-BYO ? ****design-only, DEFERRED**** | M8,M9 | ? explicit owner approval | ? | ? |

124 | ****M11**** | ****Data-flywheel harness (D12):**** on a consented adult job, ****capture**** the upload + Gemini reel/rally output ? ****reliably store**** under the per-video workspace ? ****shadow-eval**** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? ****promote**** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data\_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |

125 | ****M12**** | ****gdrive-api` (OAuth) StorageBackend (D14 Northstar):\*\*** cloud reads the source from + writes the stitched reel **\*\*back to the user's Google Drive next to the source\*\*** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |

126

127 ****Testing strategy is a first-class deliverable**** ?

[`TESTING\_STRATEGY.md`](TESTING_STRATEGY.md). ****Run logs / sample runs for posterity**** ?

[`runlogs/`](runlogs/).

128

129 **## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)***

130 ****? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:****

131 1. ? ****Cold-start ? D7**** (scale-to-zero, accept cold-start).

132 3. ? ****Pricebook ? D8**** (versioned DB table; USD internal / INR display).

133 4. ? ****Edge auth ? D9**** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).

134 5. ? ****DPA/consent ? D10**** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).

135 6. ? ****Quality floor ? D11**** + the ****data-flywheel harness D12 / M11**** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).

```

136
137     **Still open (non-gating, smaller):**
138     2. **Stitch placement:** always co-located in the detect job (simplest, one metered
unit) vs split to a cheap CPU Cloud Run service when bursty? *Default co-located unless metrics
say otherwise.*
139     7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the
source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ?
distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
```

140 8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264
(determinism/parity)?

141 9. **Input cap:** is `<2GB` a hard client reject, or accept to the 4GB upload cap and
route larger files elsewhere?

142 10. **Directory name:** `COMPUTE\_DECOUPLED\_SERVING/` (chosen) vs keep
`CLOUD\_RUN\_GPU\_SERVING/` ? confirm.

143

144 ## 9. Definition of done

145 - [] M1?M4 land (config + input + output-base + device layer), each green on golden
cross-OS + experiment no-regression.

146 - [] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical
windows + segment ranges.

147 - [] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog /
DEPLOY_REPORT** with a real sample run.

148 - [] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth
(M9) gates multi-tenant.

149 - [] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented
adult jobs; the owned model is on a measured path to the D11 floor.

150 - [] ? flip Status DONE, archive to `docs/archives/past\_projects/`.

151

152 ## 10. References

153 **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014**
(`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM\_ARCHITECTURE.md`,
`HOSTING\_PLAN.md`, `VIDEO\_LOCALITY\_MODEL.md`, `DESKTOP\_APP\_PLAN.md`,
`LOCAL\_APPLIANCE\_ARCHITECTURE.md` (khelsutra). **Archived predecessor:**
`docs/archives/past\_projects/DECOUPLED\_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-
ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code
anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet\_runner.py:281`,
`stitcher/compiler.py:303`, `segmenters/trajectory\_hybrid.py:65,237`, `backend/storage/\*`,
`backend/worker/\_\_main\_\_.py:88`, `backend/api/{uploads,job\_worker}.py`,
`backend/infrastructure/database.py`. **Design provenance:** workflow `wf\_56274ff0` (4 code-
mappers ? synthesis; 58 findings).

154

155 ---

156 ### Changelog

157 - 2026-06-21 ? created from the design workflow output; M0 in progress.

158 - 2026-06-21 ? **M0 APPROVED**; **M1 landed**
([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute\_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
change); presets touch only existing knobs (no M2 `input\_backend` / M3 output-base over-reach).
Suite 1068 green, cov 84.11%.

159 - 2026-06-21 ? **M2 landed** ([#280](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/280)): `input\_backend`/`input\_root` + URI-aware main API/CLI input via
`resolve\_input\_uri`/`acquire\_local\_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.

160 - 2026-06-21 ? **M3 landed** ([#282](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/282)): configurable output/scratch base via
`backend/utils/paths.output\_base(config)` ? the 3 segmenters' chunk/handoff/temp dirs root under
`output.output\_dir` (default `"output"`, byte-identical; default-OFF ? no Launch Report). Fixes
the latent bug where scratch ignored `--output-dir`. Suite 1114 green, cov ~84.9%.

161 - 2026-06-21 ? **M4 landed** ([#283](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/283)): `DeviceContext` + native WASB **CPU-fallback** via a new `device="auto"`
(cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade),
default unchanged. `"auto"` resolves before argv (never reaches `wasb\_infer.py`); CUDA probe

cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126 green, cov 84.96%. **? M1?M4 (the config/input/output/device layer) COMPLETE.**
162

5. assistant (2026-06-21T09:25:03.457Z)

Now let me read the TESTING_STRATEGY.md and ADR-014 to ground each gate.

6. user (2026-06-21T09:25:04.428Z)

```
1      # Testing Strategy ? Compute-Decoupled Rally Serving
2
3      > **Companion to** [`README.md`](README.md) ($7 tracker is the source of truth).
**Status:** DRAFT (2026-06-21).
4      > **Thesis:** the decoupling is only *real* if we can **prove the core is unaffected
across profiles** ? for **$0*, with no GPU/WSL/network** ? and quarantine the one thing that
genuinely can't be byte-deterministic (cross-GPU detection) to an owner-side real-hardware
layer. Tags: `[verified]` = building block already in the tree; `[design]` = to build.
5
6      ---
7
8      ## 0. The two guarantees we are testing
9      1. **Decoupling guarantee** ? the same `(input bytes + config)` produces the **same
windows + the same stitched segment ranges** whether the core runs truly-local or on Cloud Run.
*Provable for $0* because detection can be replaced by a deterministic `replay_csv` stub.
10     2. **Quality guarantee** ? real WASB detection on a real GPU is *good enough at the
rally level*. **Not** byte-deterministic across GPU architectures (L4 vs 2070 differ sub-pixel ?
684/1195 frames, per the 2026-06-20 deploy report), so it is judged at the **window/rally
level**, owner-side, on the golden corpus.
11
12     The whole strategy hangs on keeping these **separate**: CI gates #1 deterministically;
the owner gates #2 on real hardware.
13
14     ## 1. Test layers
15
16     ### L1 ? Unit (pure core) `[verified]`
17     - **What:** `trajectory_to_action_windows` determinism; `_merge_overlapping`;
`_watermark_filter` string build; `_parse_time`; **profile preset/precedence resolution** (new);
`StorageRef.parse_uri` scheme dispatch.
18     - **How:** pytest, no GPU/net/WSL; same-input?same-output; `abs-tol 1e-6` on floats.
19     - **Modes:** all (pure fns are profile-independent).
20
21     ### L2 ? Contract (seam ABCs) `[verified]`
22     - **What:** `DetectorRunner` ABC ? `run_predict` CSV schema + `healthcheck` non-raising
? across `Wasb`/`NativeWasb`/`Stub`; `StorageBackend` ABC roundtrip (`put` ? exists ? stat ? list
? fetch ? delete) across local/s3/gcs/gdrive; the new `DeviceContext` probe contract.
23     - **How:** stub runner for detection; **injected fake S3/GCS clients + gdrive=`tmp_path`
mount** (`tests/test_storage.py`); assert the CSV columns are identical across runners.
24     - **Modes:** `cpu-mock` proves the contract; local/cloud use the same code paths.
25
26     ### L3 ? Integration (GPU-free E2E) `[verified]`
27     - **What:** the full trajectory-hybrid pipeline (detect ? window ? persist ? stitch) on
the stub detector; `worker cmd_infer` fetch ? segment ? put with fake storage; chunked-upload
assemble.
28     - **How:** `RALLY_DETECTOR_IMPL=stub` (synth **and** `replay_csv`); storage fakes;
`tmp_path` workspace; assert `intervals.csv` + `report.json` + a reel are produced
deterministically.
29     - **Modes:** `cpu-mock`, but it **exercises the cloud-serving code path**
(fetch/put/dispatch) with no cloud.
30
31     ### L4 ? **Profile-PARITY (the load-bearing proof)** `[design]`
32     - **What:** the explicit decoupling guarantee ? run the **same `replay_csv` golden
trajectory** through (a) the in-process **truly-local** path and (b) the **worker + fake-`gcs`**
(cloud-serving-mock) path, and assert **byte-identical candidate windows + identical stitched
segment ranges** (and, on a fixed clip, an identical reel via ffmpeg determinism).
```



```

33 - **How:** one test parametrized over `{profile=truly-local, profile=cloud-serving-
mock}`, fed the same replay CSV + same windowing config; **diff the window list and segment
ranges exactly.** Because detection is a deterministic replay here, **any divergence is
necessarily a profile/seam leak** ? exactly what must never happen.
34 - **Modes:** truly-local vs cloud-serving (mocked). **This is the test that, if green,
proves the whole design.**
35
36 ### L5 ? E2E real-GPU (owner-side) `[design]`
37 - **What:** real WASB detection on a real GPU (local 2070 ? truly-local; GCP L4 ? cloud-
serving) end-to-end ? reel; cross-GPU parity checked at the **rally/window level** (NOT CSV
bytes).
38 - **How:** owner runs the **golden corpus** on the GPU box and on Cloud Run; compare
rally counts/windows; **smoke-test the reel** (`ffmpeg -f null -` full decode exit 0 + a 5-frame
contact sheet visually confirming real rallies). Recorded in a **DEPLOY_REPORT** under
`runlogs/`.
39 - **Modes:** truly-local + cloud-serving on **real hardware**.
40
41 ## 2. Real-GPU vs mocked ? the split
42 - **Mocked is the CI default.** The stub detector (CPU, deterministic, **byte-exact via
`replay_csv`**) + in-memory storage fakes prove **all plumbing + the profile-parity guarantee**
with **$0** and no GPU/WSL/network.
43 - **Real-GPU is owner-side only** (the golden corpus + labelled videos live with the
owner). It validates that `NativeWasbRunner` actually uses the GPU and that cross-GPU detection
diffs are **sub-pixel** ? judged at the window level, never CSV bytes.
44 - **The contract:** CI gates the deterministic post-trajectory transforms (window +
stitch) **byte-exact**; real-GPU gates that detection on the chosen GPU is **good enough at the
rally level**. Honest-limits caveat (golden $6) baked in: **green CI proves
plumbing/determinism, not field quality.**
45
46 ## 3. CI gates (extend the existing lanes)
47 Today's lanes: `ruff`, `mypy` (lenient baseline), import-smoke (no GPU stack), full
suite (offline, mocked, `--cov-fail-under`), golden-fixtures cross-OS (ubuntu/windows/macOS).
**Add:**
48 - **Profile-parity lane** ? the L4 test (truly-local vs cloud-mock identical
windows/segments).
49 - **Storage-fakes roundtrip lane** ? L2 across all four backends.
50 - **Per-PR core-touching gate** ? any PR touching detect/window/stitch **callers** (esp.
**M3**, the `output/` de-hardcode) must show: golden fixtures green on **all 3 OSes** AND
`experiment.compare(CONTROL, profile-variant)` **F1-delta ? 0** (no-regression), AND **coverage
not regressed** vs base.
51 - **No real GPU / net / WSL in CI** ? ever. Real validation is the owner's L5.
52
53 ## 4. Profile-parity ? why it's airtight
54 The decoupling claim is proven by feeding one `replay_csv` golden trajectory through
**both** the in-process path and the worker+fake-bucket path and asserting **byte-identical**
windows + segment ranges. Detection is replaced by a deterministic replay, so **the only
variable is the seam** (compute target + storage backend). If the outputs ever diverge, a
profile leaked into the core ? the precise failure D1 forbids. Real cross-GPU detection diffs
are **quarantined** to L5 and judged at the window level.
55
56 ## 5. Sample runs / runlogs for posterity (the DECOUPLED convention)
57 Each real run writes a dated report under `[runlogs/](runlogs/)` (e.g.
`DEPLOY_REPORT_{profile}_{date}.md`), mirroring
`DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md`, capturing:
58 - resources provisioned + the **exact commands**;
59 - run results ? **rally counts, parity numbers**, `gpu_active_seconds` / `wall_seconds`
/ `bytes_out`;
60 - a **reel smoke-test** ? `ffmpeg -f null -` exit 0 + a 5-frame contact sheet visually
confirming real rallies;
61 - gotchas (conda-ToS, WASB-C3, GPU stockouts, font fallback) + the **final teardown
state** ($0 audit);
62 - so the next deploy is **one-shot**.
63
64 **DoD:** `truly-local` (M5) and `cloud-serving` (M7) each get **?1 committed sample
run** here.

```

```

65
66     ## 6. Coverage map ? what each milestone must prove before merge
67     | Milestone | Must be green |
68     |---|---|
69     | M1 config | L1 preset/precedence; profile='' = today (no behaviour change) |
70     | M2 input_backend | L2 storage roundtrip (4 backends); L3 worker fetch with
local=identity |
71     | M3 output base (? core callers) | golden cross-OS byte-identical + experiment no-
regression + **L4 parity** |
72     | M4 DeviceContext | L2 device probe; cuda default unchanged; CPU-fallback unit-tested |
73     | M5 truly-local | L1?L4 all green + **owner L5 truly-local runlog** |
74     | M6 dispatch + image | mocked-dispatch tests; image builds; ffmpeg/fonts/CUDA on PATH |
75     | M7 cloud-serving | **owner L5 cloud DEPLOY_REPORT** (real L4, reel smoke-test) |
76     | M8 cost ledger | unit tests on the pricebook + aggregation; reconcile probes vs
platform billing in the runlog |
77     | M9 edge auth | per-tenant scoping tests on `user_id`; header-trust/spoofing guard test
|
78
79     ## 7. Anti-goals / pitfalls (explicit)
80     - **Don't** assert detector-CSV byte-equality across GPUs ? it *will* false-fail (D5).
81     - **Don't** add a profile `if` inside detect/window/stitch ? the parity test exists to
catch exactly that.
82     - **Don't** let `output/` de-hardcoding (M3) silently change the default layout ? keep
`output/` the default; Launch Report on any flip.
83     - **Don't** confuse truly-local's intentionally-unmetered stitch with a cost bug ? only
cloud-serving meters.
84

```

7. user (2026-06-21T09:25:05.675Z)

```

637: (owner sequencing). Seed scope below. **? REALIZED in ADR-014 (2026-06-21):** the
subproject is
639: (not just cloud serving); ADR-014 expands this 7-point seed into the M0?M10 plan.
686:## ADR-014: Compute-decoupled rally serving ? one pure core, profile-selected compute +
storage
691:`ADR-005` (permissive licensing ? engine + image stay MIT/Apache/BSD) . `ADR-008`
(train==serve featurizer single-sourcing ? the parity discipline this extends to local?cloud) .
`ADR-009` (**local-first delivery** ? origin of the truly-local profile) . `ADR-010` (DECOUPLED
DO?GCP) . `ADR-011` (**deferred M3 serving + M4 billing**; overrode "rent nothing / not a
service"; S06 gates = DPA/consent, collector `md5`, WASB-C3) . `ADR-012` (**Cloud Run+GPU =
serving model**; NVDEC lever) . `ADR-013` (**GCP sole compute stack**) . **? ADR-014** (this).
Non-ADR: realizes `PLATFORM_ARCHITECTURE.md`'s `ComputeBackend`/`StorageBackend` registries; the
truly-local profile = `VIDEO_LOCALITY_MODEL.md` L0 + `DESKTOP_APP_PLAN.md` `LocalDesktop`;
informed by the archived `DEPLOY_REPORT_GCP_2026-06-20` (cross-GPU sub-pixel parity finding).

```

8. assistant (2026-06-21T09:25:08.304Z)

Let me read the full ADR-014 section to ground the gates further.

9. user (2026-06-21T09:25:09.096Z)

```

686     ## ADR-014: Compute-decoupled rally serving ? one pure core, profile-selected compute +
storage
687
688     **Date:** 2026-06-21 . **Status:** ? **RATIFIED (owner, 2026-06-21 ? M0 design sign-off;
build M1+).** **Refines ADR-011 + ADR-012** (it *realizes* their deferred M3/M4; it does **not**
supersede them); **builds on ADR-013** (GCP sole compute stack). Full design:
`docs/COMPUTE_DECOUPLED_SERVING/`.
689
690     ### Evolution / lineage (trace the idea)
691     `ADR-005` (permissive licensing ? engine + image stay MIT/Apache/BSD) . `ADR-008`
(train==serve featurizer single-sourcing ? the parity discipline this extends to local?cloud) .
`ADR-009` (**local-first delivery** ? origin of the truly-local profile) . `ADR-010` (DECOUPLED
DO?GCP) . `ADR-011` (**deferred M3 serving + M4 billing**; overrode "rent nothing / not a

```

service"; \$06 gates = DPA/consent, collector `md5`, WASB-C3) . `ADR-012` (**Cloud Run+GPU = serving model**; NVDEC lever) . `ADR-013` (**GCP sole compute stack**) . **? ADR-014** (this). Non-ADR: realizes `PLATFORM\_ARCHITECTURE.md`'s `ComputeBackend`/`StorageBackend` registries; the truly-local profile = `VIDEO\_LOCALITY\_MODEL.md` L0 + `DESKTOP\_APP\_PLAN.md` `LocalDesktop`; informed by the archived `DEPLOY\_REPORT\_GCP\_2026-06-20` (cross-GPU sub-pixel parity finding).

692

693 ### Decision

694

695 1. The rally engine is a **PURE function of (input bytes + config) ? outputs**, in three deterministic stages ? **DETECT** (`DetectorRunner.run\_predict`?CSV), **WINDOW** (`trajectory\_to\_action\_windows`, CPU-pure), **STITCH** (`VideoCompiler.compile\_highlights`, ffmpeg) ? that are **NEVER branched on the deployment profile**.

696 2. A **`ComputeBackend` / deployment profile** decides **WHERE** the core runs (local process vs a Cloud Run GPU job); a **`StorageBackend`** decides **WHERE I/O comes from** (local FS / USB / mounted Google Drive / bucket). **Both are selected by ONE startup/setup config** (`deployment.profile` + `compute\_target`), realizing the PLATFORM_ARCHITECTURE `local|hosted|byo\_worker` registry.

697 3. Ship at least two profiles: **truly-local** (local GPU + **in-process stitch**, no cloud) and **cloud-serving** (a Cloud Run GPU job runs detect **AND** stitch), plus **cpu-mock** for CI and a reserved, **deferred** `byo\_worker` hybrid.

698 4. In cloud-serving, **STITCH runs on Cloud Run** (co-located with detect) so its CPU/wall-time + egress are **metered** into a per-job cost ledger alongside `gpu\_active\_seconds` ? stitching is compute-intensive and its cost must be accounted for. (Truly-local stitch is intentionally **un**metered.)

699 5. **Parity** is enforced at **WINDOW + STITCH** (byte-exact for a fixed trajectory); **DETECT** is byte-exact only **same-GPU** ? cross-GPU is sub-pixel-different (deploy report 2026-06-20), so it is judged at the **rally/window level**, never CSV bytes.

700 6. Land **smallest-safe-first, one PR per milestone (M0?M10)** , **DEFAULT-OFF** where a change touches core callers (e.g. the configurable output base), each guarded by golden-fixtures + a **profile-parity test** ; default flips ship a **Launch Report**.

701

702 ### Rationale

703

704 - The seams already exist: `\_resolve\_runner` is the single compute seam, the `StorageBackend` ABC + 4 backends + the worker fetch/put are built, and the stub detector (`replay\_csv`) gives a **\$0 deterministic** core. Realizing a profile system is **composition, not a rewrite**.

705 - Separating **WHERE** (ComputeBackend, coarse, per-job) from **HOW** (DeviceContext, fine, per-GPU) from **WHAT-bytes** (StorageBackend) keeps each axis independently testable and avoids the main trap ? conflating them.

706 - **Metering stitch on Cloud Run** is the only way the owner's real serving cost (GPU + CPU re-encode + egress) is attributable per job; stitch-on-laptop hides it.

707 - Aligns with ADR-012 (GPU-native) + ADR-013 (GCP sole stack) + the 2026-06-20 deploy report; the truly-local profile preserves the ADR-009 privacy/offline self-host story.

708

709 ### Consequences

710

711 - **Positive:** one core, provably identical across modes; CI proves the decoupling for **\$0** (stub + storage fakes + a profile-parity test); new compute targets (byo_worker, SkyPilot, Modal) slot into the enum without core changes; **cost becomes visible per job** (incl. stitch).

712 - **Cost/new surface:** a new `deployment`+`compute\_target` config surface, a Cloud Run **dispatch shim**, a **containerized GPU+ffmpeg worker image**, a **cost-ledger migration**, and **edge auth** must all be built + maintained. Cross-GPU detection is **not** byte-identical ? parity asserted at the window level. The `byo\_worker` tier carries a real ops/support burden (deferred, power-user only).

713 - **Risk-managed:** every core-touching change is default-OFF + golden/parity-gated + Launch-Reported; real-GPU runs are **owner-side** with a budget guard and a posterity DEPLOY_REPORT (`docs/COMPUTE\_DECOUPLED\_SERVING/runlogs/`).

714 - **Owner gates ? the 5 gating ones LOCKED 2026-06-21** (project doc \$2 **D7?D12**): D7 cold-start = **scale-to-zero** ; D8 pricebook = **versioned DB table, USD-internal/INR-display** ; D9 edge-auth = **Cloudflare Access** (verify Cf-Access JWT, lock ingress); D10 = **free-tier data-for-reels trade** (adults-only, derived-data, counsel-gated to M9); D11 quality-floor = **Gemini-ON until the owned model clears the floor** + D12 the **data-flywheel harness** (capture?store?eval?goldenize). Plus **3 more locked at M0 sign-off (2026-06-21, \$2 D13?D15):**

****D13**** profile is user-switchable anytime (advertised feature; equivalent output, not bit-identical cross-GPU); ****D14 NORTHSTAR**** = mobile/cloud-native point-at-Google-Drive ? reel back in Drive next to the source (implies a new ****`gdrive-api` OAuth**** backend = M12; resolves the gdrive-in-cloud question); ****D15**** auto-detect SUGGESTS, the user CONFIRMS ? execution (esp. cloud spend) is never a surprise. Still open (non-gating): stitch co-located vs split; NVDEC vs libx264; the `<2GB` cap; the dir name.

715

10. assistant (2026-06-21T09:25:12.406Z)

I now have all the grounding I need from the project doc, testing strategy, and ADR-014. Let me verify a couple of the code anchors referenced in the design to make claims precise about what code can be written now.

11. user (2026-06-21T09:25:15.004Z)

```
backend/api/job_worker.py
backend/config/models.py
backend/config/presets.py
backend/infrastructure/database.py
backend/infrastructure/execution_policy.py
backend/main.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/storage/base.py
backend/storage/ref.py
backend/worker/__main__.py
```

12. user (2026-06-21T09:25:16.060Z)

```
__init__.py
__pycache__
base.py
gcs.py
gdrive.py
local.py
ref.py
s3.py
--- JOBS ---
221:      # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
224:      # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
227:      self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id TEXT")
228:      self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
229:      # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
232:      "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
233:      "WHERE user_id IS NOT NULL")
236:      "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
240:      # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
247:      #   user_id           ? owner of the model (NULL allowed = the local install)
```

13. assistant (2026-06-21T09:25:19.934Z)

I have enough grounding. The seams (StorageBackend ABC + 4 backends, job table with user_id columns from v4, execution policy, presets) all exist as the design claims. Let me check the presets file briefly to confirm what M5's preset wiring would build on.

14. user (2026-06-21T09:25:21.823Z)

```
4:config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
30:# Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to warn
31:# when config.json explicitly overrides compute_target into a state inconsistent with the
profile.
33:    "truly-local": "local_gpu",
```

```

34:     "cloud-serving": "cloud_run",
35:     "cpu-mock": "cpu_mock",
39:# that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}``
plus
43:PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44:     "truly-local": {
45:         "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
51:     "cloud-serving": {
52:         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
57:     "cpu-mock": {
58:         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
68:     return bool(profile) and profile in PROFILE_PRESETS
74:     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
104:     - ``K_SERVICE`` set (Cloud Run)          ? ``"cloud-serving"``
105:     - ``CI`` truthy                          ? ``"cpu-mock"``
106:     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
110:     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
115:         return "cloud-serving"
117:         return "cpu-mock"
119:         return "truly-local"

```

15. assistant (2026-06-21T09:25:25.628Z)

I have all the grounding I need. The presets module already has the profile enum, compute_target mapping, preset bundles, and auto-detect. M5 builds the startup-check + e2e wiring on top of this existing M1 scaffolding. Now I'll compile the structured output mapping all 8 milestones plus the 4 still-open S8 design questions.

16. user (2026-06-21T09:27:31.917Z)

Structured output provided successfully

17. assistant (2026-06-21T09:27:49.530Z)

I've mapped all 12 gates (8 milestones M5-M12 + 4 still-open S8 design questions) for the serving track, grounded in the project doc, testing strategy, and ADR-014, and verified the key code seams exist.

Key findings on the Claude-doable-NOW vs owner-prerequisite split:

```

**Highest unblock value (owner decision frees large independent Claude PRs):**
- **M5 (truly-local):** Almost entirely Claude-doable now ? the M1 preset scaffolding already
exists (`backend/config/presets.py:27,33,43,104`); Claude can land the preset wiring + startup
checks + the load-bearing L4 parity test + L1?L3 coverage. Only the committed real-GPU runlog
needs the owner.
- **M6 (dispatch shim + image):** The shim + Dockerfile + mocked-dispatch tests are fully
writable behind default-OFF (the job control plane + `WAIT_AND_RESUME` carry verbatim). Owner
prerequisite is purely GCP spend to actually deploy.
- **M8 (cost ledger):** Nearly all code lands now with a seeded placeholder pricebook (the
`user_version` migration ladder + v4 `user_id` precedent exist at `database.py:221-236`). Owner
only supplies real GCP prices + a reconcile probe.
- **M9 (edge auth):** Cf-Access JWT verification + per-tenant scoping (v4 columns already exist)
are default-OFF code; only the counsel-reviewed ToS/DPA + the real CF tenant config are owner-
gated.

```

```

**Medium value:** M11 (flywheel harness mechanics buildable against fixtures, gated behind M9
consent) and M12 (`gdrive_api` StorageBackend against a fake Drive client; only the real OAuth
app is owner-gated).

```

```

**Run-execution only:** M7 (real L4 e2e + DEPLOY_REPORT) ? Claude pre-stages signed-URL delivery
+ the smoke-test/contact-sheet harness so the owner run is one-shot.

```

```

**Cheap design picks (all four still-open S8 questions are effectively pre-decided by the doc's

```

defaults):** Q2 stitch placement ? co-located (D4 already locks it); Q8 ? libx264 for parity-safe stitch encode, NVDEC decode optional; Q9 ? hard-reject at <2GB (matches shipped M2); Q10 ? confirm `COMPUTE\_DECOUPLED\_SERVING/` (already the name everywhere). None block code structurally ? each is a one-word owner confirm that lets M6/M7 build without ambiguity.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\COMPUTE\_DECOUPLED\_SERVING\README.md`, `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\COMPUTE\_DECOUPLED\_SERVING\TESTING\_STRATEGY.md`, `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\decisions\DECISIONS.md` (ADR-014, lines 686-715),
`C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\presets.py`,
`C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py`,
`C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\`.